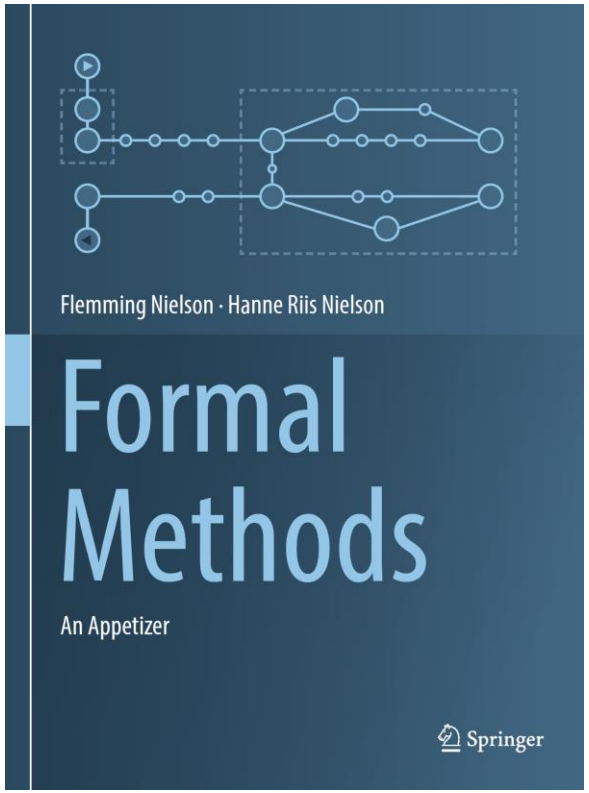
02141 – Computer Science Modelling

Formal Methods 03

Alberto Lluch Lafuente

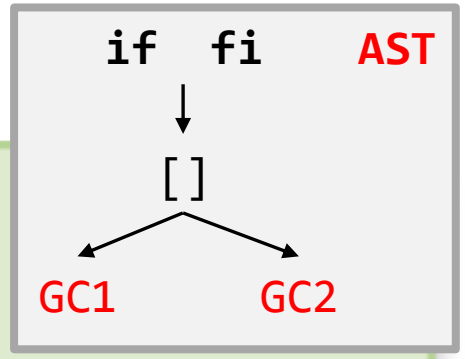
(based on slides by Christoph Matheja)

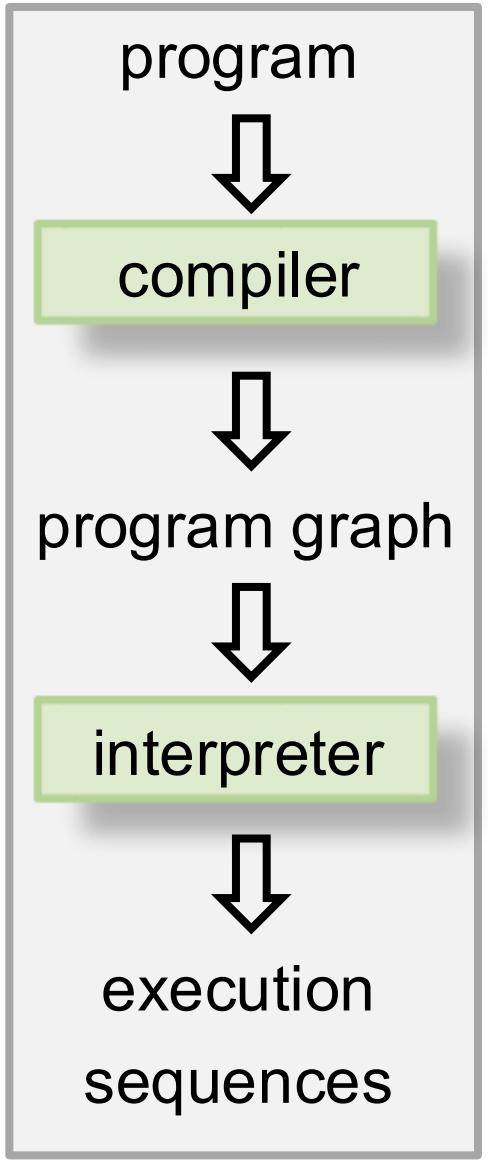


Previously…

Syntax of the Guarded Command Language

```
C  ::= x := a | skip | C1; C2 | if GC fi | do GC od
GC ::= b -> C | GC1 [] GC2
a  ::= n | x | a1 + a2 | a1 - a2 | a1 * a2 | a1 / a2 | a1 ^ a2
b  ::= true | a1 = a2 | a1 > a2 | a1 >= a2 | ¬b1 | b1 ∧ b2 | b1 && b2
```





Construction of program graphs

source node

$$\textbf{edges}(q_\circ \rightsquigarrow q_\bullet)[\![\,\mathtt{C}\,]\!] \quad \subseteq \quad \mathbf{Q} \times \mathbf{Act} \times \mathbf{Q}$$

target node

Semantic functions

$\mathcal{S}[\![\alpha]\!] : \mathbf{Mem} \hookrightarrow \mathbf{Mem}$

$\mathcal{A}[\![\mathtt{a}]\!] : \mathbf{Mem} \hookrightarrow \mathbf{Int}$

$\mathcal{B}[\![\mathtt{b}]\!] : \mathbf{Mem} \hookrightarrow \mathbf{Bool}$

Language Extensions

How do we incorporate more interesting programming language features?

```
i:=0;
x:=0;
y:=0;
do i<10 -> if A[i]>=0 -> x:=x+A[i];
                          i:=i+1
           [] A[i]<0 -> i:=i+1;
                        break
           fi;
           y:=y+1
od
```

```
i:=0;
try
  do A[i]=x -> throw yes
  [] !(A[i]=x) -> if i<9 -> i:=i+1
                  [] i>=9 -> throw no
                  fi
  od
catch yes: x:=1
   [] no: x:=2
yrt
```

Agenda – Extensions

1. Arrays

2. Deterministic-By-Construction Semantics

3. “Proof of the Day”

We will <u>modify existing concepts today</u> – no definition is entirely new

Questions to ask yourself:

- Can you read the <u>formal</u> definitions?
- Can you follow the rationale?
- Could you make similar changes?

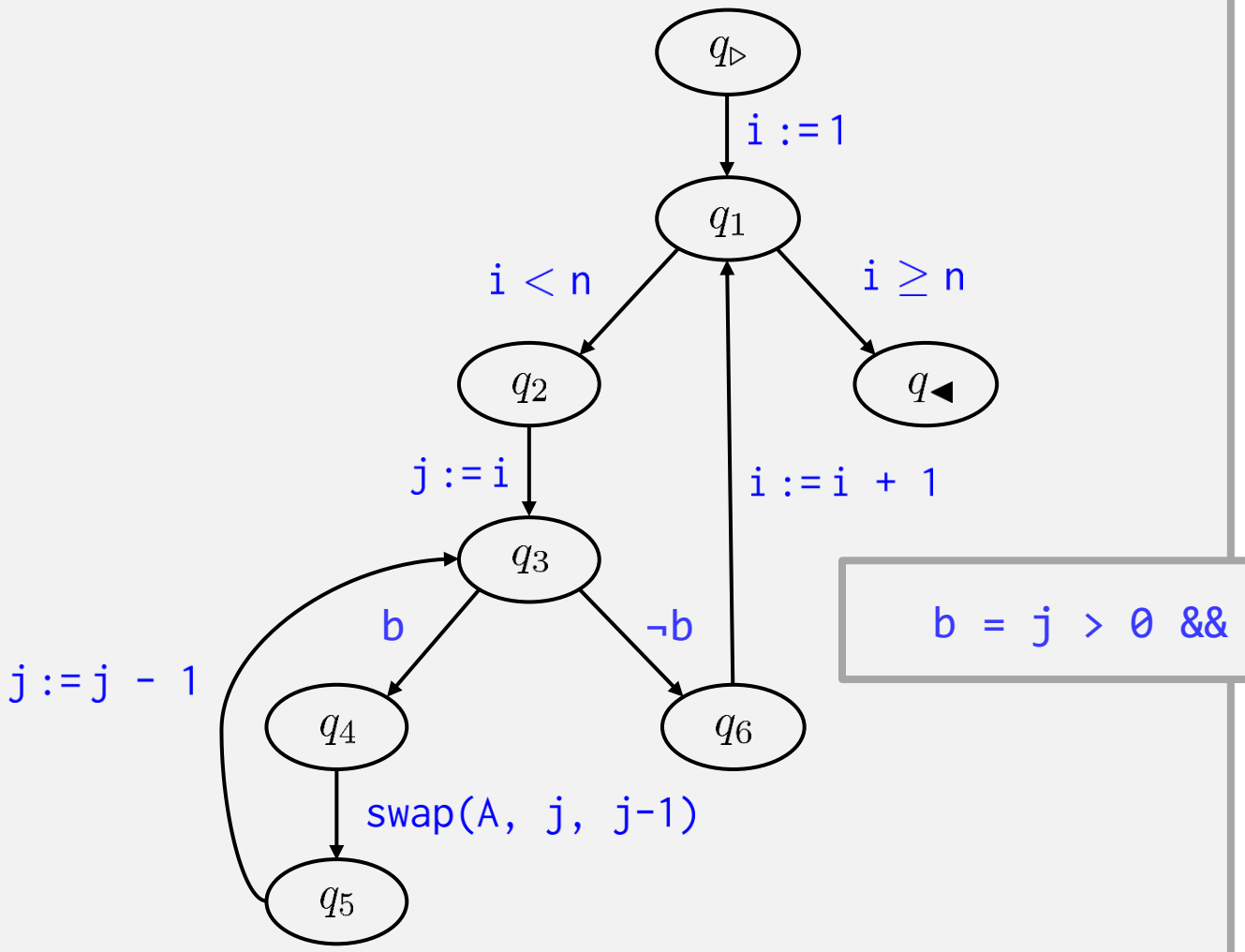
To test your “formal literacy”.

Do ask questions if confused ☺

02141 – Computer Science Modelling

EXTENSIONS: ARRAYS

Example: Insertion Sort




```
b = j > 0 && A[j-1] > A[j]
```

- **New features:**
  - fixed-size array `A`
  - `swap` method
  - array lookups `A[j]` may fail if `j` is out of bounds
  - array update `A[i] := a`

- **Goal:** extend the Guarded Command Language to account for these features

Syntax of GCL programs with arrays

Syntax of the Guarded Command Language with arrays

```
C  ::= x := a | skip | C1; C2 | if GC fi | do GC od                    (Commands)
             | A[a1] := a2 | swap(A,a1,a2)

GC ::= b -> C | GC1 [] GC2                                              (Guarded Commands)

a ::= n | x | a1 + a2 | a1 - a2 | a1 * a2 | a1 / a2 | a1 ^ a2      (Arithmetic Expressions)
        | A[a]

b ::= true | a1 = a2 | a1 > a2 | a1 >= a2 | ¬b1 | b1 ∧ b2 | b1 && b2     (Boolean expressions)
```

`A` is an array of size `length(A)` taken from the set **Arr**

Program Graphs for GCL programs with arrays

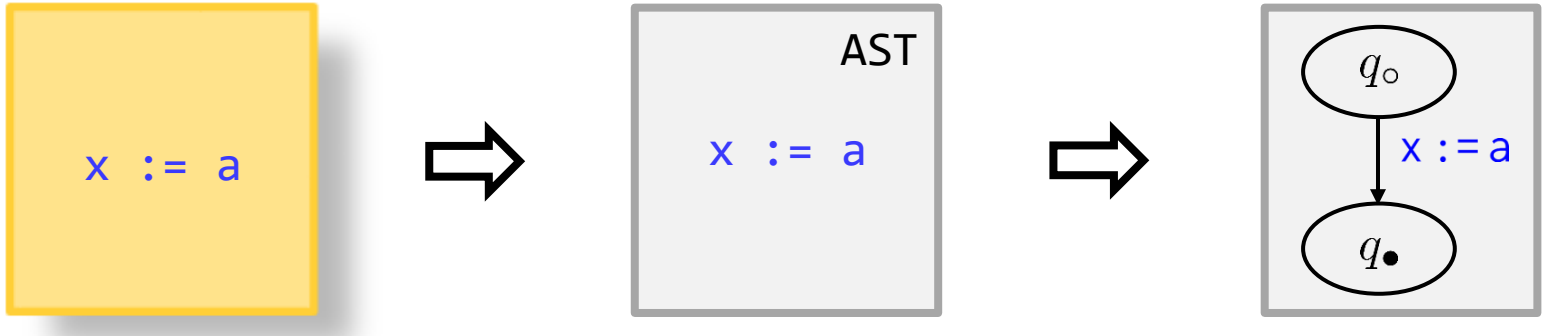
source node

command

$$\textbf{edges}(q_\circ \rightsquigarrow q_\bullet)[\![\, \mathtt{C} \,]\!] \quad \subseteq \quad \mathbf{Q} \times \mathbf{Act} \times \mathbf{Q}$$

set of program graph edges

target node



$$\textbf{edges}(q_\circ \rightsquigarrow q_\bullet)[\![\, \mathtt{x := a} \,]\!] \quad = \quad \{\, (q_\circ, \mathtt{x := a}, q_\bullet) \,\}$$

Program Graphs for GCL programs with arrays

source node

command

$$\textbf{edges}(q_\circ \rightsquigarrow q_\bullet)[\![\texttt{C}]\!] \quad \subseteq \quad \textbf{Q} \times \textbf{Act} \times \textbf{Q}$$

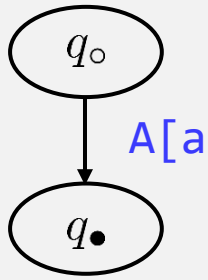
set of program graph edges

target node

```
A[a1] := a2
```

AST

```
A[a1] := a2
```



$$\textbf{edges}(q_\circ \rightsquigarrow q_\bullet)[\![\texttt{A[a1]:=a2}]\!] \quad = \quad \{\,(q_\circ, \texttt{A[a1]:=a2}, q_\bullet)\,\}$$

Recap: memories and semantic functions

Definition. A **semantic function**

$$\mathcal{S}[\![\cdot]\!] : \mathbf{Act} \to (\mathbf{Mem} \hookrightarrow \mathbf{Mem})$$

maps every action $\alpha \in \mathbf{Act}$ to a *partial* function

$$\mathbf{Mem} \hookrightarrow \mathbf{Mem}$$

specifying the effect of performing action α on any given memory.

Mem is the set of all memories

So far: memory for variables

$$\sigma : \mathbf{Var} \to \mathbf{Int}$$

v	σ(v)
x	12
y	5

Now: incorporate arrays A in **Arr**

Memories for Variables and Arrays

Main idea: treat every *array element* `A[i]` like a *variable* in **Var**

Definition. A **memory for variables and arrays** is a function

$$\sigma : \quad (\mathbf{Var} \cup \{\, \mathtt{A[i]} \mid \mathtt{A} \in \mathbf{Arr},\ 0 \leq \mathtt{i} < \mathrm{length}(\mathtt{A}) \,\}) \quad \rightarrow \quad \mathbf{Int}$$

that assigns an integer value to every variable and every array element `A[i]`.

All previous definitions also work for our new set Mem of memories

Definition (memory update).

$$\sigma[\mathtt{A[i]} \mapsto \mathtt{v}](\mathtt{y}) \;=\; \begin{cases} \mathtt{v}, & \text{if } \mathtt{y} = \mathtt{A[i]} \\ \sigma(\mathtt{y}), & \text{if } \mathtt{y} \neq \mathtt{A[i]} \end{cases}$$

v	σ(v)
n	4
i	1
j	2
A[0]	4
A[1]	2
A[2]	17
A[3]	9

Quiz: To what does the expression evaluate?

$$\mathcal{A}[\![\texttt{A[A[i]]}]\!](\sigma)$$

v	σ(v)
n	4
i	1
j	3
A[0]	4
A[1]	2
A[2]	17
A[3]	9

2

4

17

undefined

Quiz: To what does the expression evaluate?

$$\mathcal{A}[\![\texttt{A[A[i]]}]\!](\sigma)$$

v	σ(v)
n	4
i	4
j	3
A[0]	4
A[1]	2
A[2]	17
A[3]	9

2

4

17

undefined

Evaluation of Array Expressions

$$\mathcal{A}[\![\texttt{A[a]}]\!](\sigma) = \begin{cases} \sigma(\texttt{A}[u]), & \text{if } u = \mathcal{A}[\![\texttt{a}]\!](\sigma) \text{ defined}, \quad \texttt{A}[u] \in \text{dom}(\sigma) \\ \text{undefined}, & \text{otherwise} \end{cases}$$

"is defined for memory σ"

needed to deal with out-of-bounds cases

v	σ(v)
n	4
i	1
j	3
A[0]	4
A[1]	2
A[2]	17
A[3]	9

$$\xrightarrow{\mathcal{A}[\![\texttt{A[A[i]]}]\!](\sigma)} \quad 17$$

Quiz: Which memory is returned?

$$\mathcal{S}[\![\texttt{A[i+1] := 42}]\!](\sigma)$$

v	σ(v)
n	4
i	1
j	3
A[0]	4
A[1]	2
A[2]	17
A[3]	9

v	σ1(v)
n	4
i	1
j	3
A[0]	4
A[1]	42
A[2]	17
A[3]	9

v	σ2(v)
n	4
i	1
j	3
A[0]	4
A[1]	2
A[2]	42
A[3]	9

v	σ3(v)
n	4
i	1
j	3
A[0]	4
A[1]	42
A[2]	42
A[3]	42

undefined

Quiz: Which memory is returned?

$$\mathcal{S}[\![\texttt{A[i+1] := 42}]\!](\sigma)$$

v	σ(v)
n	4
i	3
j	3
A[0]	4
A[1]	2
A[2]	17
A[3]	9

v	σ1(v)
n	4
i	3
j	3
A[0]	4
A[1]	42
A[2]	17
A[3]	9

v	σ2(v)
n	4
i	3
j	3
A[0]	4
A[1]	2
A[2]	42
A[3]	9

v	σ3(v)
n	4
i	3
j	3
A[0]	4
A[1]	42
A[2]	42
A[3]	42

undefined

Semantics of Array Updates

Read ',' as "and"

$$\mathcal{S}[\![\texttt{A[a1] := a2}]\!](\sigma) = \begin{cases} \sigma[\texttt{A}[u] \mapsto v], & \text{if } u = \mathcal{A}[\![\texttt{a1}]\!]\sigma \text{ defined,} \\ & \quad v = \mathcal{A}[\![\texttt{a2}]\!]\sigma \text{ defined,} \\ & \quad \texttt{A}[u] \in \text{dom}(\sigma) \\ \text{undefined,} & \text{otherwise} \end{cases}$$

v	σ(v)
n	4
i	1
j	3
A[0]	4
A[1]	2
A[2]	17
A[3]	9

$$\xrightarrow{\mathcal{S}[\![\texttt{A[i+1] := 42}]\!](\sigma)}$$

v	σ'(v)
n	4
i	1
j	3
A[0]	4
A[1]	2
A[2]	42
A[3]	9

Wrap-Up

- Formalizing the semantics of advanced language features may require more involved definitions of memories
- Example in this lecture: arrays
- Advanced examples: pointers, objects, heap-based data structures
- It is often sufficient to formalize a minimal set of statements
  - Add other statements as “syntactic sugar”
  - Simulate advanced statements (if acceptable)

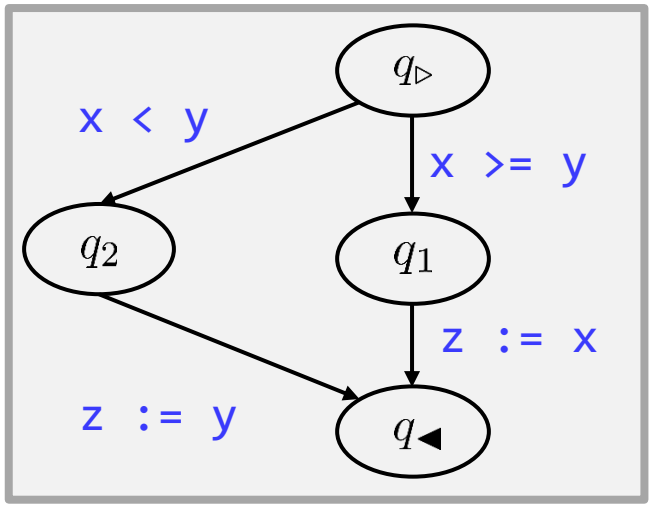
02141 – Computer Science Modelling

DETERMINISTIC-BY-CONSTRUCTION SEMANTICS

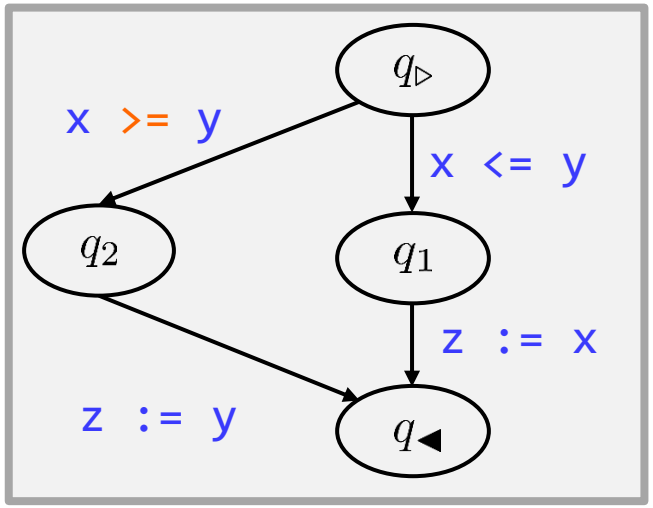
Example

Deterministic system: for every initial memory, there is at most one complete execution sequence.

```
if x >= y -> z := x
[] x < y  -> z := y
fi // deterministic
```



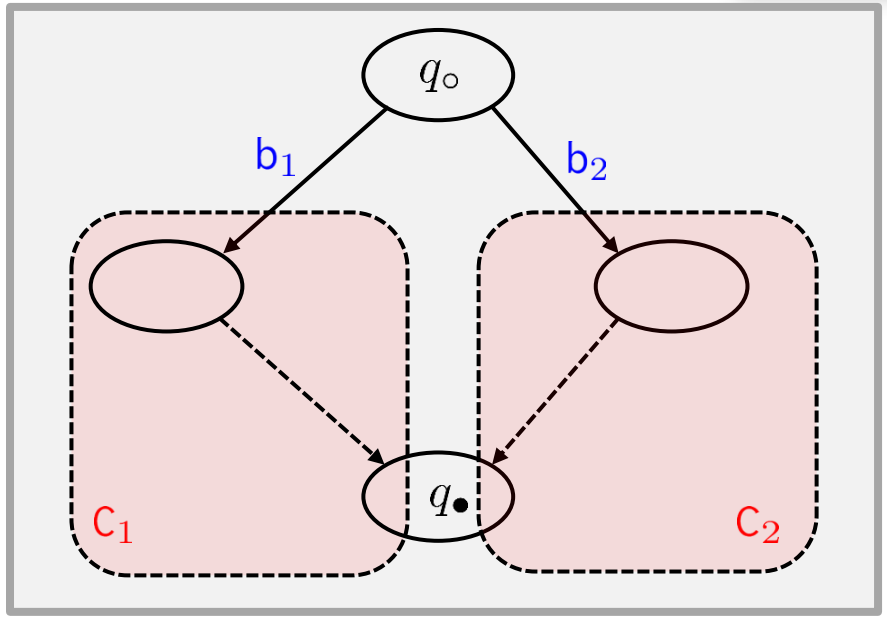

```
if x >= y -> z := x
[] x <= y -> z := y
fi // non-deterministic
```

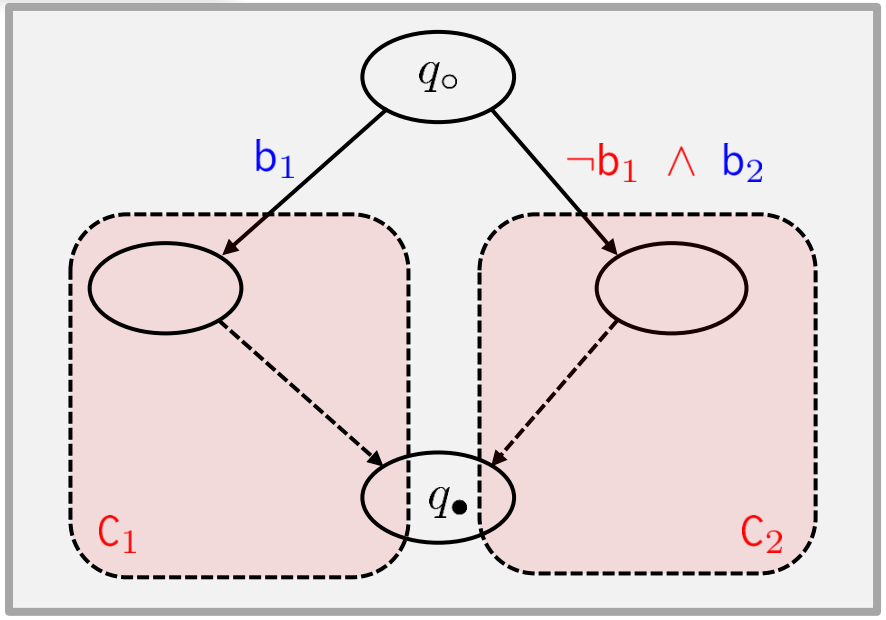


A Deterministic Semantics – Main Idea

- **Goal:** change semantics such that we always get a deterministic system
- **Main idea:** always execute the first possible guarded command

$$\textbf{if}\ \{\, b_1 \rightarrow C_1 \,\}\ [\,]\ \{\, b_2 \rightarrow C_2 \,\}\ \textbf{fi}$$





deterministic version

A Deterministic Semantics – Formally

set of edges

Definition (deterministic edges).

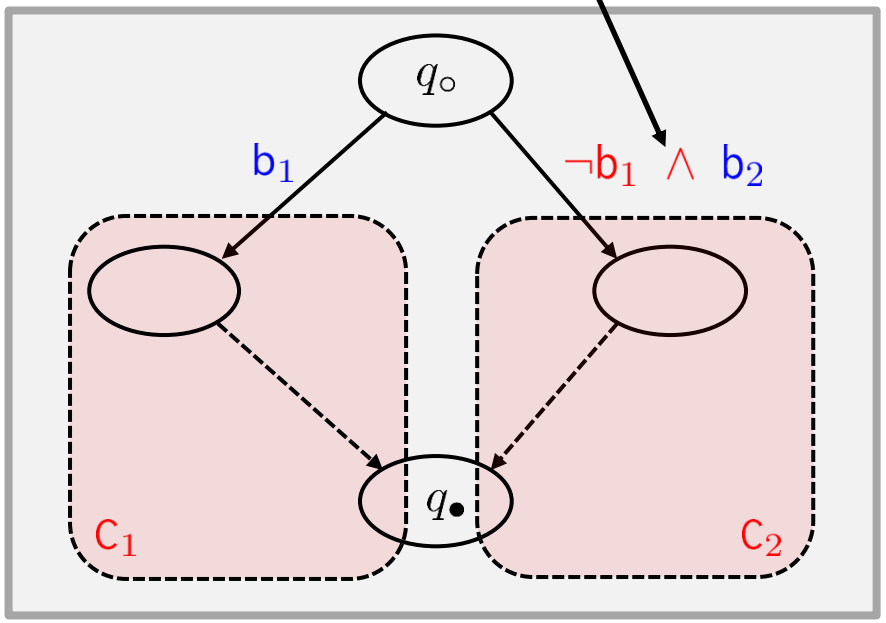
$$\textbf{edges}_{\text{det}}(q_\circ \rightsquigarrow q_\bullet)[\![\mathtt{GC}]\!](d) : \mathsf{Pow}(\textbf{Q} \times \textbf{Act} \times \textbf{Q}) \times \mathtt{BExpr}$$

disjunction of previous guards

Boolean expression

For guarded commands, **edges** returns a set of edges *and* a Boolean expression

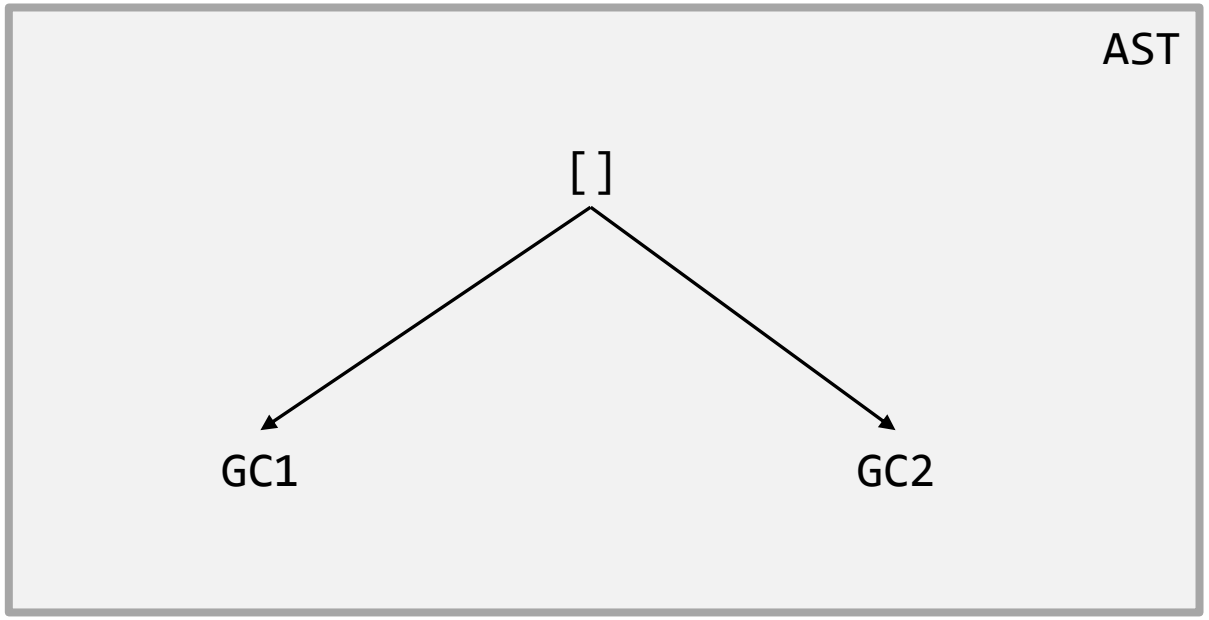
"no previous guard holds"

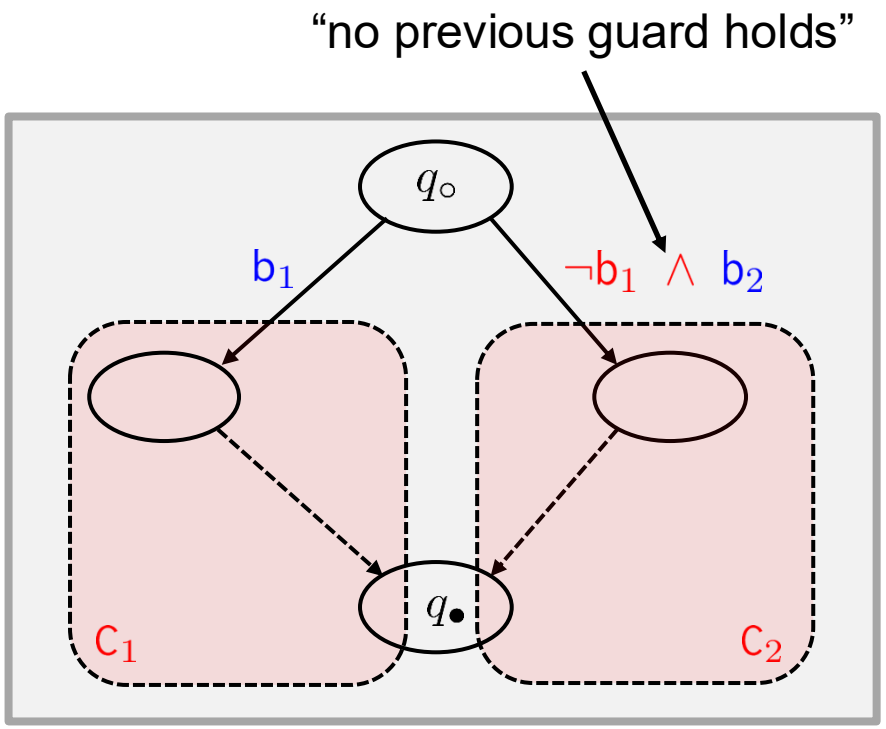


$$\textbf{edges}_{\text{det}}(q_\circ \rightsquigarrow q_\bullet)[\![b \to C]\!](d) \;=\; (\{(q_\circ, b \wedge \neg d, q)\} \cup \textbf{edges}(q \rightsquigarrow q_\bullet)[\![C]\!],\; b \vee d)$$

where q is a fresh node

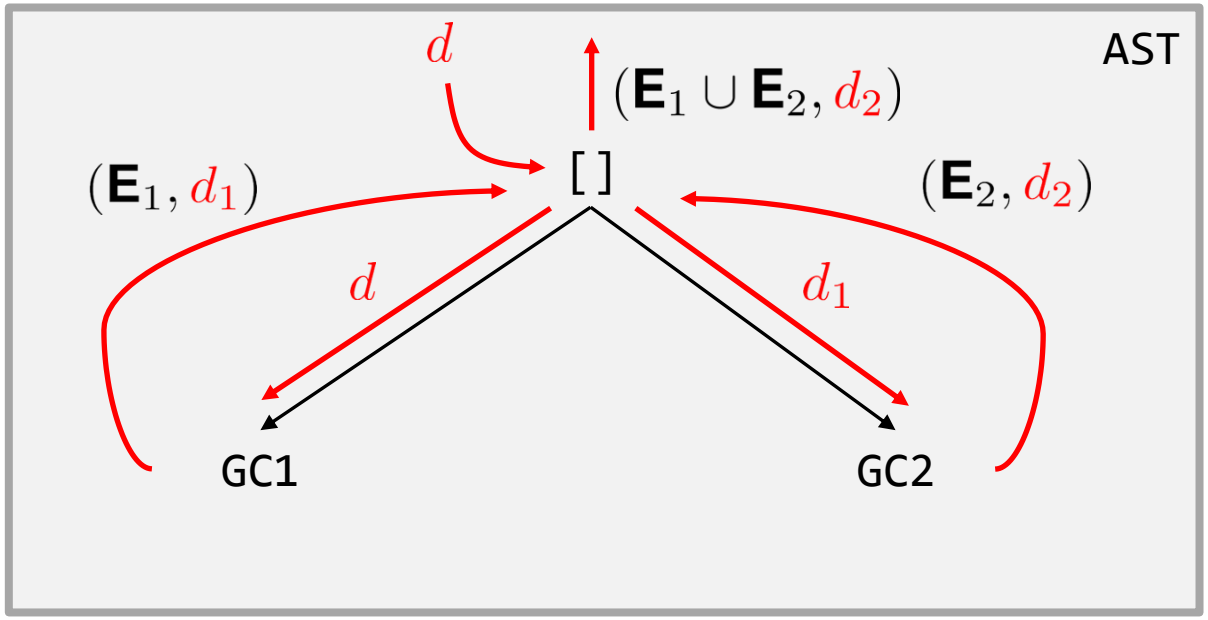
A Deterministic Semantics – Formally

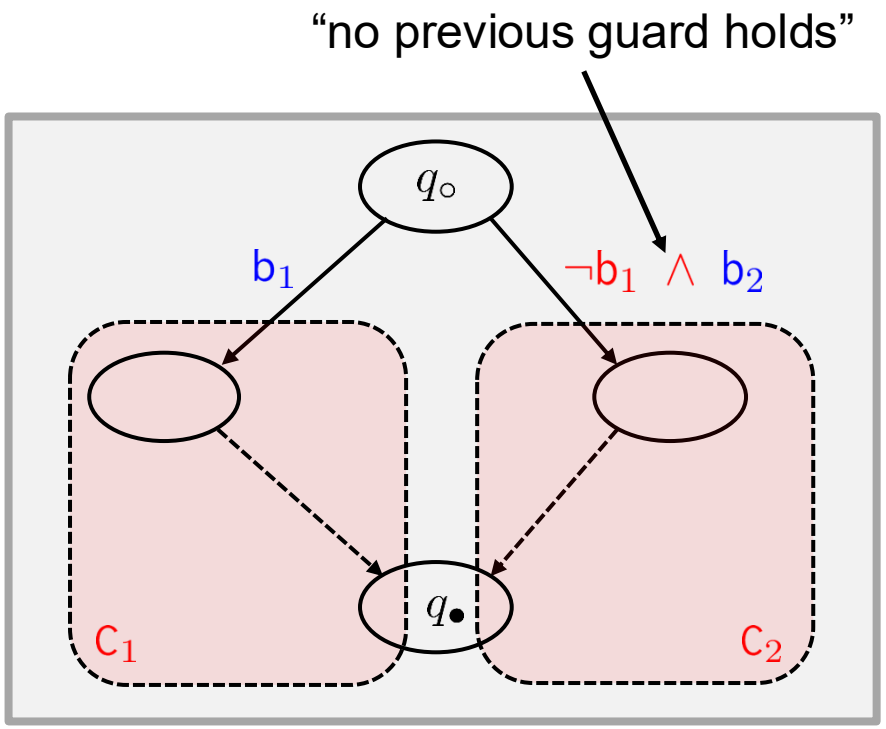




$$\textbf{edges}_{\text{det}}(q_\circ \rightsquigarrow q_\bullet)[\![\{\,\mathtt{GC}_1\,\}\ [\,]\ \{\,\mathtt{GC}_2\,\}]\!](d) \;=\; (\mathbf{E}_1 \cup \mathbf{E}_2, d_2), \text{ where}$$
$$(\mathbf{E}_1, d_1) \;=\; \textbf{edges}_{\text{det}}(q_\circ \rightsquigarrow q_\bullet)[\![\mathtt{GC}_1]\!](d),$$
$$(\mathbf{E}_2, d_2) \;=\; \textbf{edges}_{\text{det}}(q_\circ \rightsquigarrow q_\bullet)[\![\mathtt{GC}_2]\!](d_1)$$

A Deterministic Semantics – Formally





$$\mathbf{edges}_{\text{det}}(q_\circ \rightsquigarrow q_\bullet)[\![\{\,\mathtt{GC}_1\,\}\ [\ ]\ \{\,\mathtt{GC}_2\,\}]\!](d) \;=\; (\mathbf{E}_1 \cup \mathbf{E}_2, d_2), \text{ where}$$

$$(\mathbf{E}_1, d_1) \;=\; \mathbf{edges}_{\text{det}}(q_\circ \rightsquigarrow q_\bullet)[\![\mathtt{GC}_1]\!](d),$$

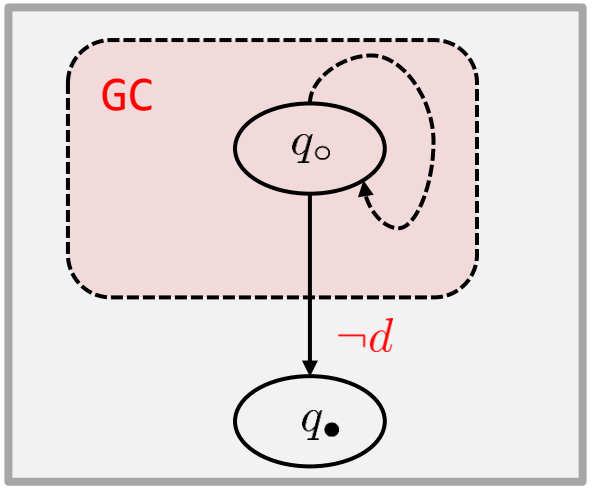
$$(\mathbf{E}_2, d_2) \;=\; \mathbf{edges}_{\text{det}}(q_\circ \rightsquigarrow q_\bullet)[\![\mathtt{GC}_2]\!](d_1)$$

A Deterministic Semantics – Formally

$$\textbf{edges}(q_\circ \rightsquigarrow q_\bullet)[\![\texttt{C}]\!] : \mathrm{Pow}(\mathbf{Q} \times \mathbf{Act} \times \mathbf{Q})$$

$$\textbf{edges}(q_\circ \rightsquigarrow q_\bullet)[\![\texttt{if GC fi}]\!] = \mathbf{E}, \text{ where } (\mathbf{E}, d) = \textbf{edges}_{\text{det}}(q_\circ \rightsquigarrow q_\bullet)[\![\texttt{GC}]\!](\texttt{false})$$

$$\textbf{edges}(q_\circ \rightsquigarrow q_\bullet)[\![\texttt{do GC od}]\!] = \mathbf{E} \cup \{ (q_\circ, \neg d, q_\bullet) \}, \text{ where}$$
$$(\mathbf{E}, d) = \textbf{edges}_{\text{det}}(q_\circ \rightsquigarrow q_\circ)[\![\texttt{GC}]\!](\texttt{false})$$

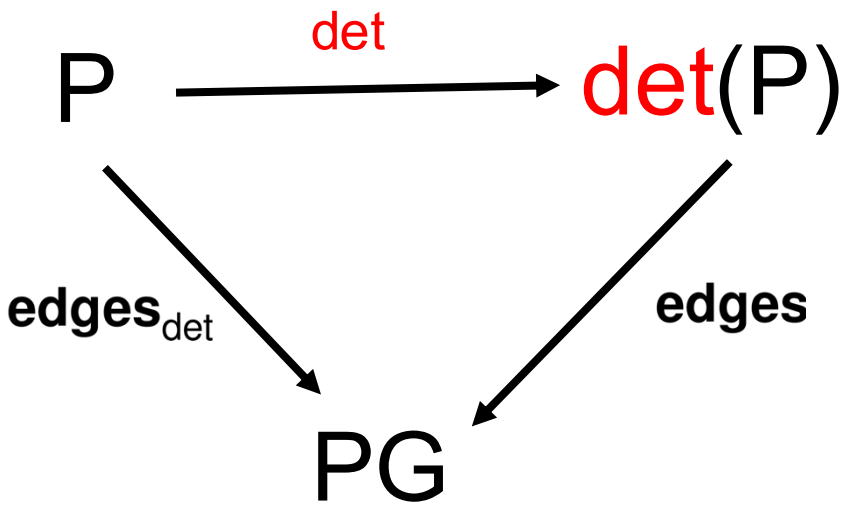


$$\text{done}[\![\texttt{b -> C}]\!] = \neg \texttt{b}$$
$$\text{done}[\![\{\ \texttt{GC}_1\ \}\ [\ ]\ \{\ \texttt{GC}_2\ \}]\!] = \text{done}[\![\texttt{GC}_1]\!] \wedge \text{done}[\![\texttt{GC}_2]\!]$$

$$\text{done}[\![\texttt{GC}]\!] = \neg d$$

A Deterministic Semantics – Alternative approach

We could also define a program-to-program function det such that det(P) does the job of making P deterministic. Goal:



How???

Structural recursion / induction to the rescue… Let's try!

Exercise (formalmethods.dk/FM4FUN)

- Use FM4FUN to construct the program graph for the following program

```
if x >= 10 -> y := 10
[] x >= 5  -> y := 5
[] x >= 0  -> y := 0
fi
```

- How many complete execution sequences are there (depending on x) for...
  - ... when using the non-deterministic semantics?
  - ... when using the deterministic semantics?

Wrap-Up

- Formalizing the semantics of a programming language involves many design choices, e.g. evaluation orders

set of edges

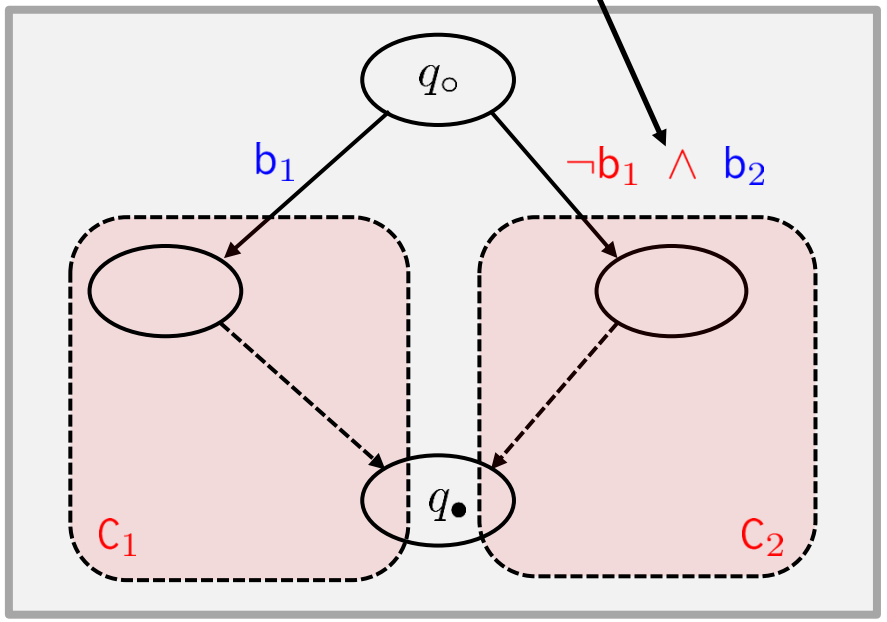
Definition (deterministic edges).

$$\mathbf{edges}_{\text{det}}(q_\circ \rightsquigarrow q_\bullet)[\![\mathtt{GC}]\!](d) : \mathsf{Pow}(\mathbf{Q} \times \mathbf{Act} \times \mathbf{Q}) \times \mathtt{BExpr}$$

disjunction of previous guards

previous guards + our guard

"no previous guard holds"



RECURSIVE DEFINITIONS, INDUCTIVE PROOFS

02141 – Computer Science Modelling

RECAP: STRUCTURAL INDUCTION

The Principle of Structural Induction

- **Given:** an inductive set S , i.e. all elements are either
  - atomic elements $s \in S$ or
  - composed elements $f(s_1, \ldots, s_n) \in S$ obtained from applying one of finitely many functions f to elements $s_1, \ldots, s_n \in S$.

- **Claim to show:** For all $s \in S$, the property $P(s)$ holds.

- **Proof: we need to show that**
  - **(induction base)** $P(s)$ holds for every atomic element $s \in S$, and
  - **(induction hypothesis)** if $P(s_1), \ldots, P(s_n)$ hold for arbitrary, but fixed, elements $s_1, \ldots, s_n \in S$
  - **(induction step)** then $P(\, f(s_1, \ldots, s_n)\,)$ also holds for every operation f .

Example: Natural induction

- The set of natural numbers is an inductive set, where
  - 0 is the only atomic element and
  - if n is a natural number, then $n + 1$ is a composed element.

- **Claim to show:** $P(n)$ holds for all natural numbers n.

- **Proof: we need to show that**
  - **(induction base)** $P(0)$ holds and
  - **(induction hypothesis)** if $P(n)$ holds for an arbitrary, but fixed, natural number n,
  - **(induction step)** then $P(n+1)$ also holds.

Example: the structure of words

- The set of words over an alphabet Σ is an inductive set, where
  - ε is the only atomic element and
  - if w is a finite word and $a \in \Sigma$, then the word aw is a composed element.

- **Claim to show:** *P(w)* holds for all words *w.*

- **Proof: we need to show that**
  - **(induction base)** $P(\varepsilon)$ holds and $P(a)$ holds for every $a \in \Sigma$
  - **(induction hypothesis)** if $P(w)$ holds for an arbitrary, but fixed, word w,
  - **(induction step)** then $P(aw)$ also holds for every $a \in \Sigma$

Example: the structure of GCL programs

- The set of all GCL commands C and guarded commands GC is an (mutually) inductive set, where
  - $skip, x := a$ are atomic elements
  - if $C1$ and $C2$ are commands, then $C1; C2$ is a composed command
  - If GC is a guarded command, then $if\ GC\ fi$ is a composed command
  - … (one case per production in the grammar!)

- **Claim to show:** *P(C)* holds for all commands *C.*

- **Proof: we need to show that**
  - **(induction base)** *P*($skip$) and *P*($x := a$) hold
  - **(induction hypothesis)** if *P*($C1$), *P*($C2$), *P*($GC1$), *P*($GC2$) hold
  - **(induction step1)** then *P*($C1; C2$)
  - **(induction step2)** then *P*($if\ GC1\ fi$)
  - …

The variables of an arithmetic expression

set of variables that appear in expression a

Simplified Syntax of Arithmetic Expressions a.

```
a ::= n                    (int constants are atoms)
    | x                    (variables are atoms)
    | a1 + a2              (addition)
    | a1 * a2              (multiplication)
```

a	$Var(\mathtt{a})$
n	$\emptyset$
x	$\{\mathtt{x}\}$
a1 + a2	$Var(\mathtt{a1}) \cup Var(\mathtt{a2})$
a1 * a2	$Var(\mathtt{a1}) \cup Var(\mathtt{a2})$

Claim P(a): for every simplified arithmetic expression a and all memories σ, σ' with $\sigma(x) = \sigma'(x)$ for all variables $x \in Var(a)$, we have $\mathrm{A}[\![a]\!](\sigma) = \mathcal{A}[\![a]\!](\sigma')$.

Reminder: structural induction on the syntax of (simplified) arithmetic expressions

- **Goal:** property P(a) holds for all expressions a
- **Induction base:** P(n) and P(x) hold
- **Induction hypothesis (I.H.):** P(a1) and P(a2) hold for arbitrary, but fixed, expressions a1, a2
- **Induction step:** if we assume I.H., then P(a1 + a2) and P(a1 * a2) hold

Proof

The case P(n): trivial because $A[\![n]\!](\sigma) = n = \mathcal{A}[\![n]\!](\sigma')$ for all memories σ, σ'

The case P(x): assume $\sigma(y) = \sigma'(y)$ for all variables $y \in Var(x) = \{x\}$

$$A[\![x]\!](\sigma) = \sigma(x) = \sigma'(x) = A[\![x]\!](\sigma')$$

The case P(a1 + a2): assume $\sigma(y) = \sigma'(y)$ for all variables $y \in Var(a1 + a2)$

$$\begin{aligned}
& A[\![a1 + a2]\!](\sigma) \\
= {} & A[\![a1]\!](\sigma) + A[\![a2]\!](\sigma) && \text{(by definition)} \\
= {} & A[\![a1]\!](\sigma') + A[\![a2]\!](\sigma') && \text{(by I.H.)} \\
= {} & A[\![a1 + a2]\!](\sigma') && \text{(by definition)}
\end{aligned}$$

The case P(a1 * a2) is analogous to the case P(a1 + a2).

What next?

- A closer look at grammars and non-regular languages
- Parsing: how do we get from program source code to ASTs?
- How can we use our program semantics to reason about properties of programs?

Exercise Class

- Today's exercises: DTU Learn > Content > FM3
  - You can always ask us for feedback and/or help

Thanks for today!

02141 – Computer Science Modelling

BONUS MATERIAL BELOW, JUST IF YOU ARE CURIOUS ☺

02141 – Computer Science Modelling

EXTENSIONS: BREAK & CONTINUE

Examples

```
do i < 10 ->
  if A[i] = 42 -> break
  [] A[i] != 42 -> i := i + 1
   fi
od;
return := i
```

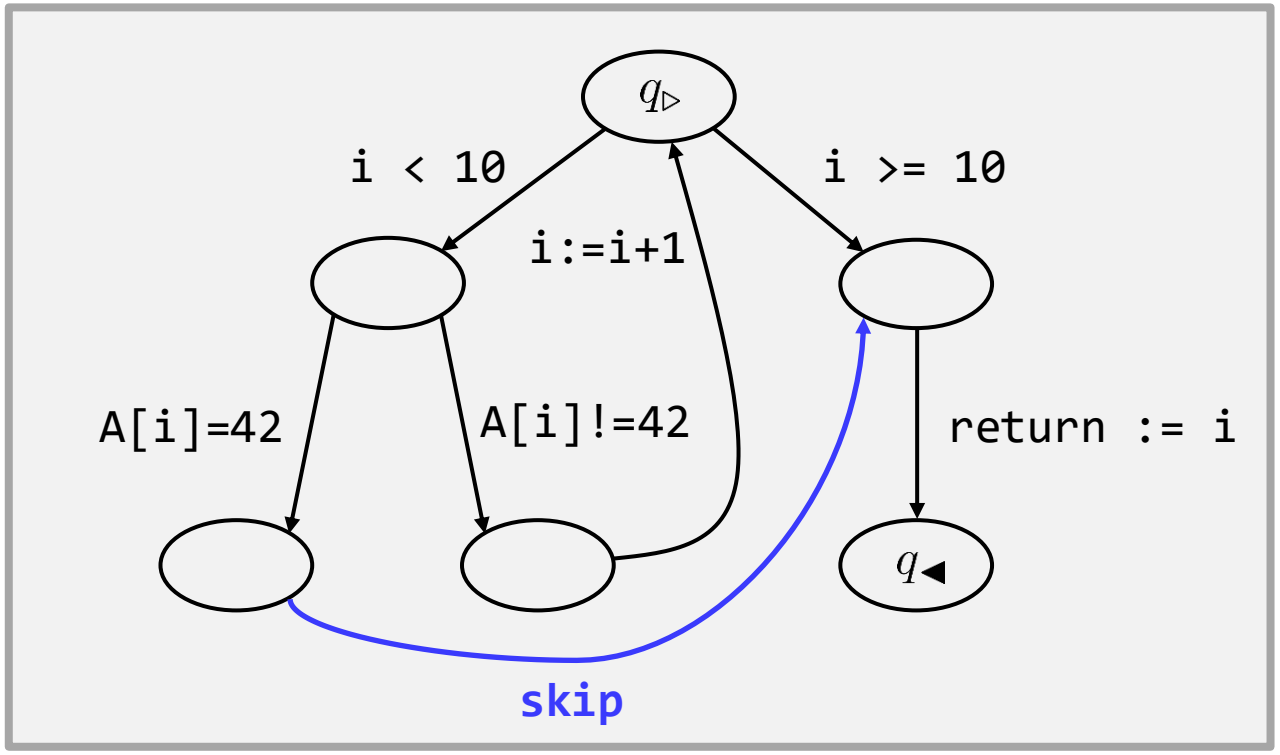
```
do i < 10 ->
   if A[i] <= 0 -> continue
   [] A[i] > 0 -> skip
   fi;
   sum := sum + A[i];
   i := i + 1
od
```

- `break` inside of loops transfers control to the end of the innermost loop
- `continue` inside of loops transfers control to the beginning of the innermost loop

The break Command

- `break` inside of loops transfers control to the end of the innermost loop
- We model break via a `skip` action; alternatives?

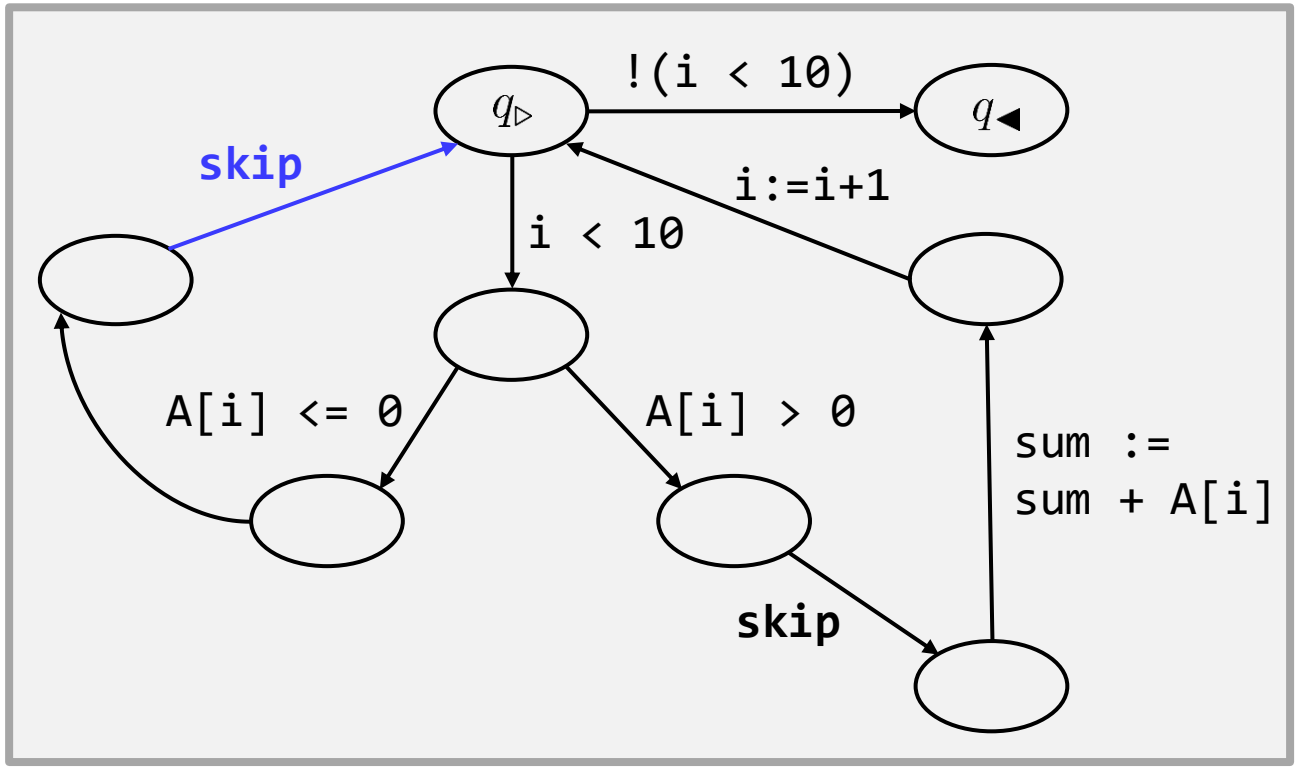
```
do i < 10 ->
  if A[i] = 42 -> break
  [] A[i] != 42 -> i := i + 1
   fi
od;
return := i
```



The continue Command

- `continue` inside of loops transfers control to the beginning of the innermost loop

```
do i < 10 ->
   if A[i] <= 0 -> continue
   [] A[i] > 0 -> skip
   fi;
   sum := sum + A[i];
   i := i + 1
od
```



formalmethods.dk/FM4FUN: break vs. continue vs. skip

- Construct program graphs for variants of the given program using break and continue instead of skip. Why are the program graphs as constructed?
- The program is found in Examples ➔ *add and count entries of an array*

```
i := 0;
x := 0;
y := 0;
do i < 10 -> if A[i] >= 0 -> x := x + A[i];
                             i := i + 1
           [] A[i] < 0 -> i := i + 1;
                          break
           fi;
           y := y + 1
od
```

Formalizing Break and Continue

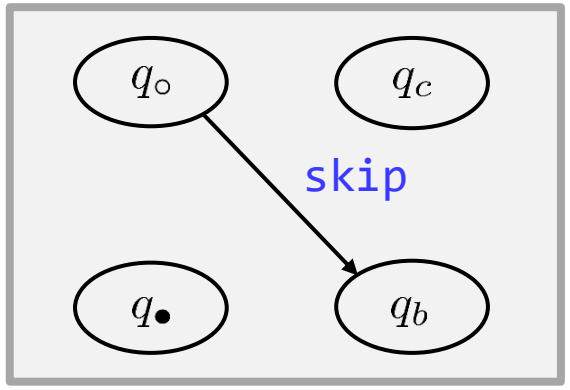
source node

target nodes for break and continue

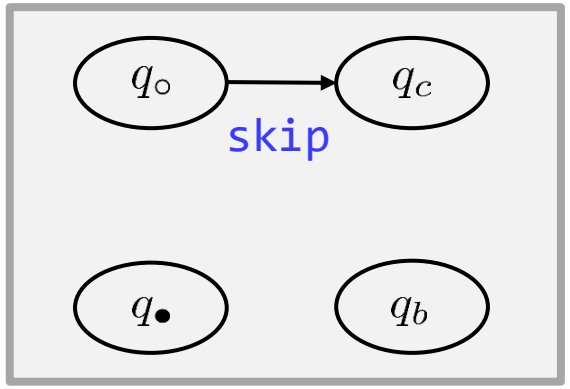
$$\textbf{edges}_{\text{bc}}(q_\circ \rightsquigarrow q_\bullet)[\![\texttt{C}]\!](q_b, q_c) \quad \subseteq \quad \textbf{Q} \times \textbf{Act} \times \textbf{Q}$$

target node

$$\textbf{edges}_{\text{bc}}(q_\circ \rightsquigarrow q_\bullet)[\![\texttt{break}]\!](q_b, q_c) \;=\; \{\,(q_\circ, \textbf{skip}, q_b)\,\}$$



$$\textbf{edges}_{\text{bc}}(q_\circ \rightsquigarrow q_\bullet)[\![\texttt{continue}]\!](q_b, q_c) \;=\; \{\,(q_\circ, \textbf{skip}, q_c)\,\}$$



Formalizing Break and Continue

source node

target nodes for break and continue

$$\textbf{edges}_{\text{bc}}(q_\circ \rightsquigarrow q_\bullet)[\![\,\texttt{C}\,]\!](q_b, q_c) \quad \subseteq \quad \textbf{Q} \times \textbf{Act} \times \textbf{Q}$$

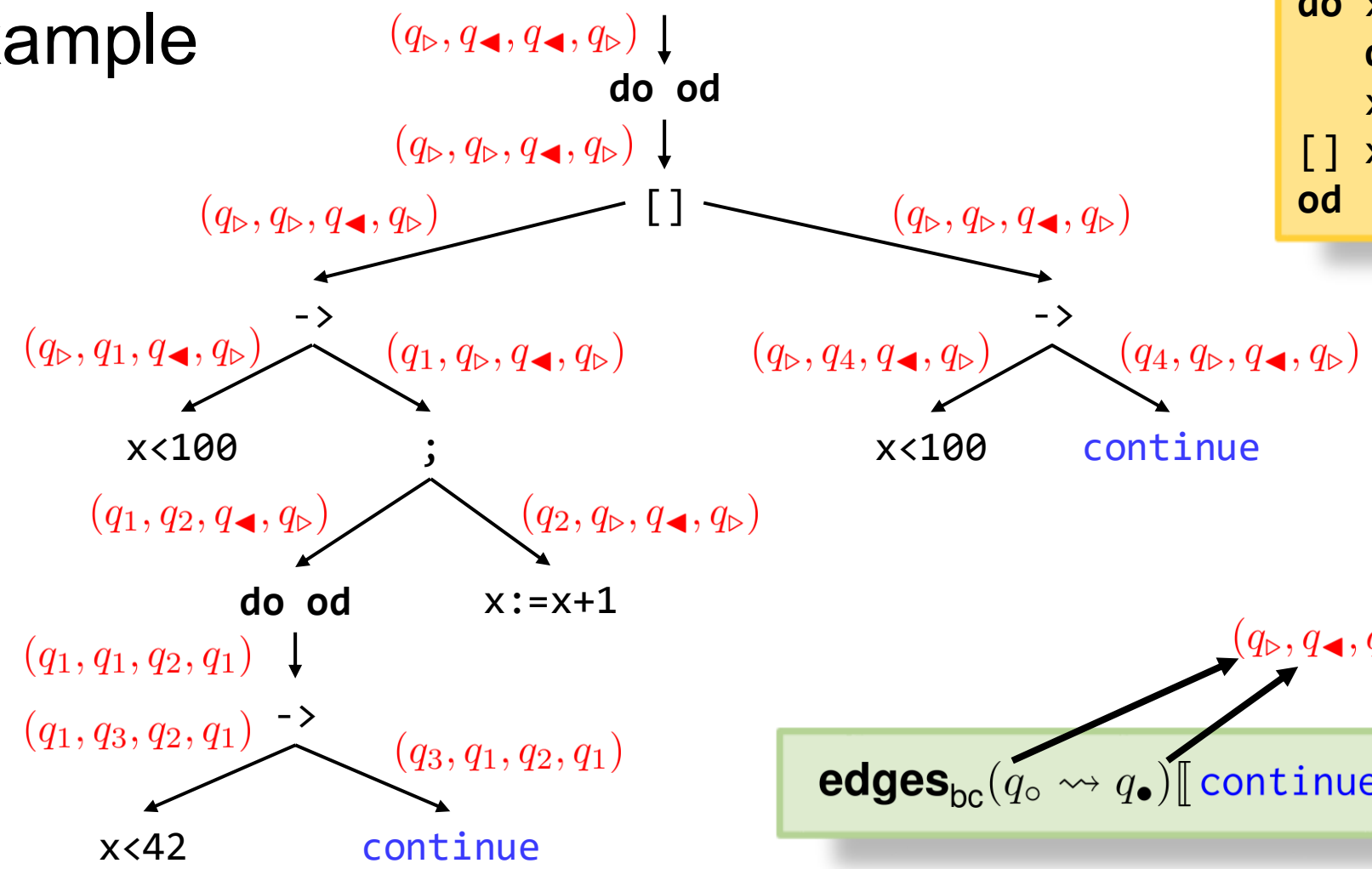
target node

`break` goes to the loop's end

$$\textbf{edges}_{\text{bc}}(q_\circ \rightsquigarrow q_\bullet)[\![\,\textbf{do}\ \texttt{GC}\ \textbf{od}\,]\!](q_b, q_c) \quad = \quad \textbf{edges}_{\text{bc}}(q_\circ \rightsquigarrow q_\circ)[\![\,\texttt{GC}\,]\!](q_\bullet, q_\circ) \ \cup \ \{\,(q_\circ, \text{done}[\![\texttt{GC}]\!], q_\bullet)\,\}$$

`continue` goes to the loop's beginning

Example




```
do x < 100 ->
   do x < 42 -> continue od;
   x := x + 1
[] x < 100 -> continue
od
```

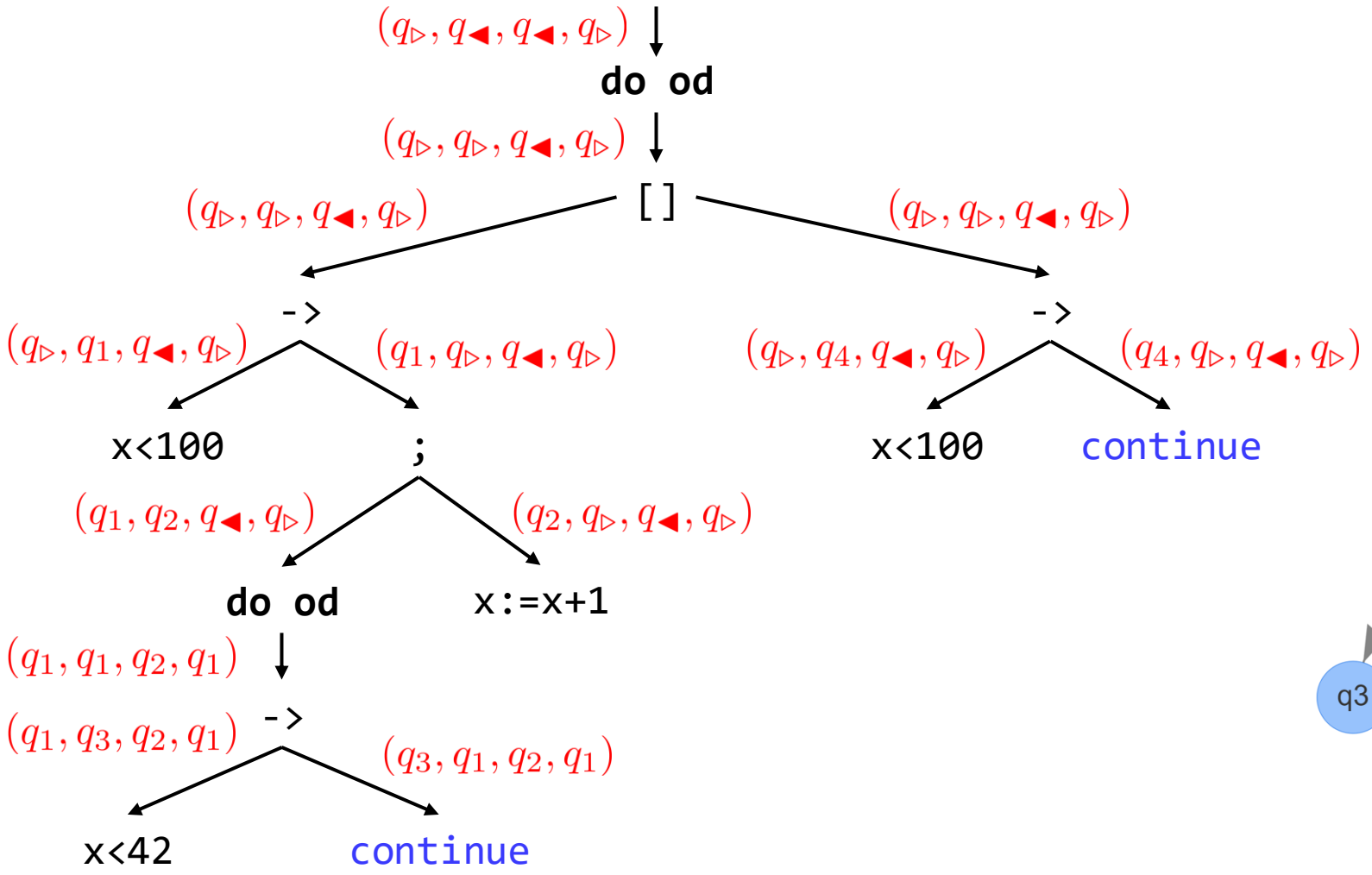
$$\textbf{edges}_{\text{bc}}(q_\circ \rightsquigarrow q_\bullet)[\![\,\texttt{continue}\,]\!](q_b, q_c) \;=\; \{\,(q_\circ, \textbf{skip}, q_c)\,\}$$

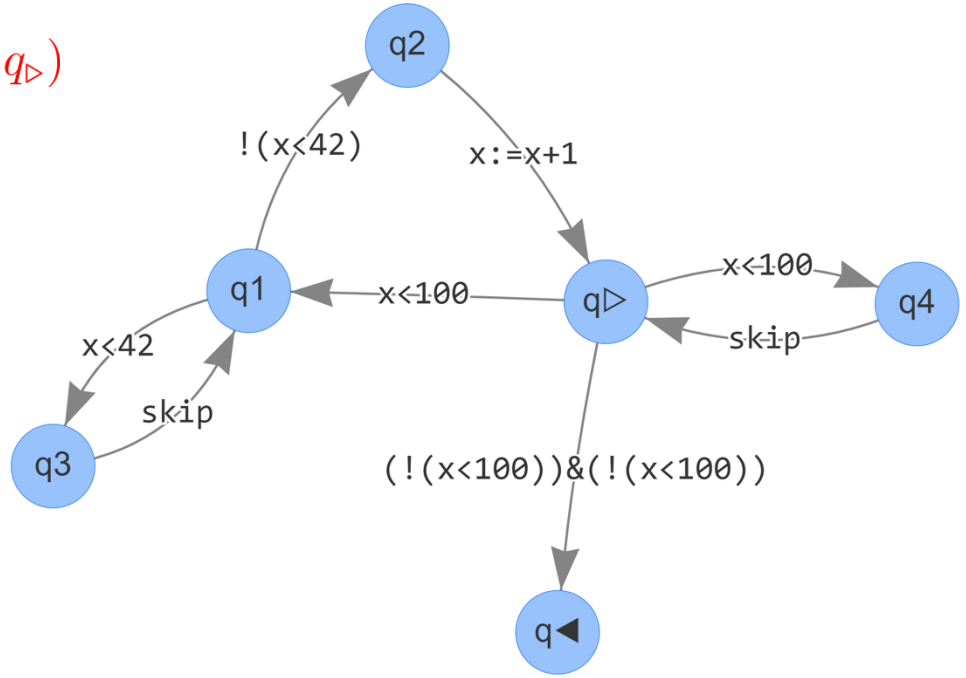
$(q_\triangleright, q_\blacktriangleleft, q_\blacktriangleleft, q_\triangleright)$

$$\textbf{edges}_{\text{bc}}(q_\circ \rightsquigarrow q_\bullet)[\![\,\textbf{do}\ \texttt{GC}\ \textbf{od}\,]\!](q_b, q_c) \;=\; \textbf{edges}_{\text{bc}}(q_\circ \rightsquigarrow q_\circ)[\![\,\texttt{GC}\,]\!](q_\bullet, q_\circ) \;\cup\; \{\,(q_\circ, \text{done}[\![\texttt{GC}]\!], q_\bullet)\,\}$$

Example

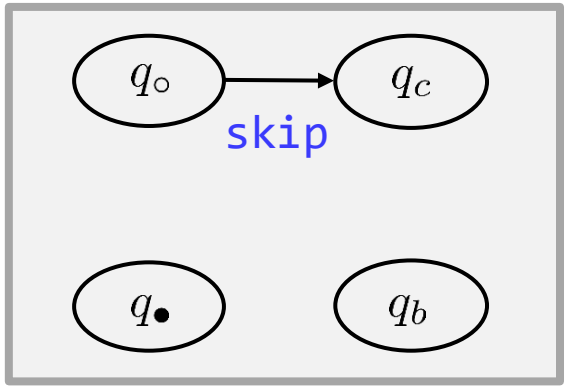
```
do x < 100 ->
   do x < 42 -> continue od;
   x := x + 1
[] x < 100 -> continue
od
```





Wrap-Up

- Formalizing the semantics of control structures may involve passing around many potential target nodes while traversing the AST



source node

target nodes for break and continue

$$\textbf{edges}_{\text{bc}}(q_\circ \rightsquigarrow q_\bullet)[\![\texttt{C}]\!](q_b, q_c) \quad \subseteq \quad \textbf{Q} \times \textbf{Act} \times \textbf{Q}$$

target node

02141 – Computer Science Modelling

EXTENSIONS: EXCEPTIONS

Example

FM4FUN: searching for a value

```
i:=0;
try
  do A[i]=x -> throw yes
  [] !(A[i]=x) -> if i<9 -> i:=i+1
                  [] i>=9 -> throw no
                  fi
  od
catch yes: x:=1
   [] no: x:=2
yrt
```

$$\mathbf{edges}_{\mathsf{x}}(q_\circ \rightsquigarrow q_\bullet)[\![\, \mathtt{C} \,]\!](\gamma) \quad \subseteq \quad \mathbf{Q} \times \mathbf{Act} \times \mathbf{Q}$$

handler environment γ: Exceptions $\hookrightarrow$ **Q**
assigning target nodes to exceptions

Extended GCL Syntax

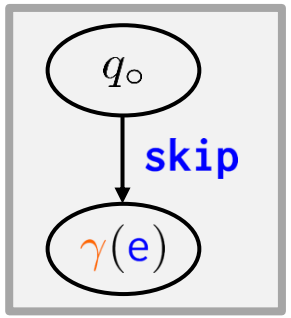
```
C  ::= ...
     | throw e
     | try C catch HC yrt

HC ::= e: C
     |  HC1 [] HC2
```

- `throw e`: move to target node $\gamma(\mathtt{e})$
- `try C catch HC yrt`: use `HC` to update handler environment
- Just as for variables, we assume a set of exception names

Throwing Exceptions

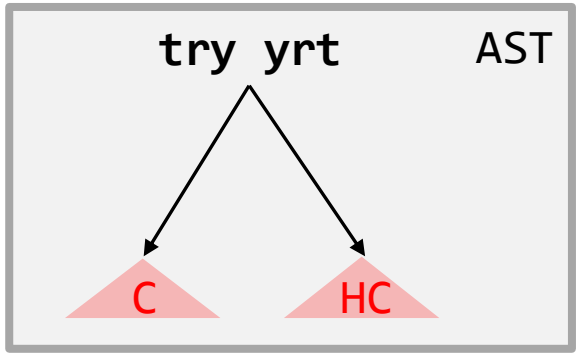
$$\textbf{edges}_{\mathsf{X}}(q_\circ \rightsquigarrow q_\bullet)[\![\, \texttt{throw e} \,]\!](\gamma) \;=\; \{\, (q_\circ, \textbf{skip}, \gamma(\texttt{e})) \mid \texttt{e} \in \mathsf{dom}(\gamma) \,\}$$

γ: Exceptions $\hookrightarrow$ **Q**



Updating the Handler Environment γ: Exceptions $\hookrightarrow$ Q

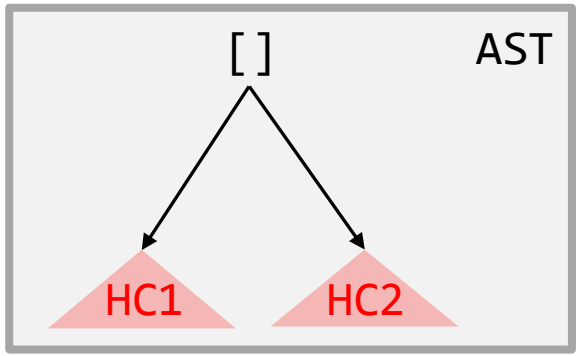
$$\begin{aligned} & \textbf{edges}_{\mathsf{x}}(q_\circ \rightsquigarrow q_\bullet)[\![\texttt{try}\ \texttt{C}\ \texttt{catch}\ \texttt{HC}\ \texttt{yrt}]\!](\gamma) \\ = \ & \mathbf{E}' \cup \textbf{edges}_{\mathsf{x}}(q_\circ \rightsquigarrow q_\bullet)[\![\texttt{C}]\!](\gamma') \\ & \text{where } (\mathbf{E}', \gamma') = \textbf{edges}_{\mathsf{x2}}(\rightsquigarrow q_\bullet)[\![\texttt{HC}]\!](\gamma) \end{aligned}$$



$$\textbf{edges}_{\mathsf{x2}}(\rightsquigarrow q_\bullet)[\![\texttt{e:}\ \texttt{C}]\!](\gamma) \ = \ (\textbf{edges}_{\mathsf{x}}(q \rightsquigarrow q_\bullet)[\![\texttt{C}]\!](\gamma),\ \gamma[\texttt{e} \mapsto q])$$

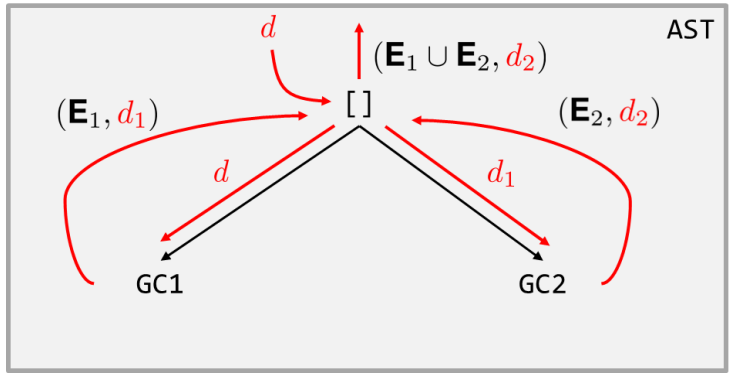
where q is a fresh node

$$\begin{aligned} & \textbf{edges}_{\mathsf{x2}}(\rightsquigarrow q_\bullet)[\![\{\ \texttt{HC1}\ \}\ [\ ]\ \{\ \texttt{HC2}\ \}]\!](\gamma) \ = \ (\mathbf{E}_1 \cup \mathbf{E}_2, \gamma_2) \\ & \quad \text{where } (\mathbf{E}_1, \gamma_1) = \textbf{edges}_{\mathsf{x2}}(\rightsquigarrow q_\bullet)[\![\texttt{HC1}]\!](\gamma), \\ & \quad \text{and } (\mathbf{E}_2, \gamma_2) = \textbf{edges}_{\mathsf{x2}}(\rightsquigarrow q_\bullet)[\![\texttt{HC2}]\!](\gamma_1) \end{aligned}$$

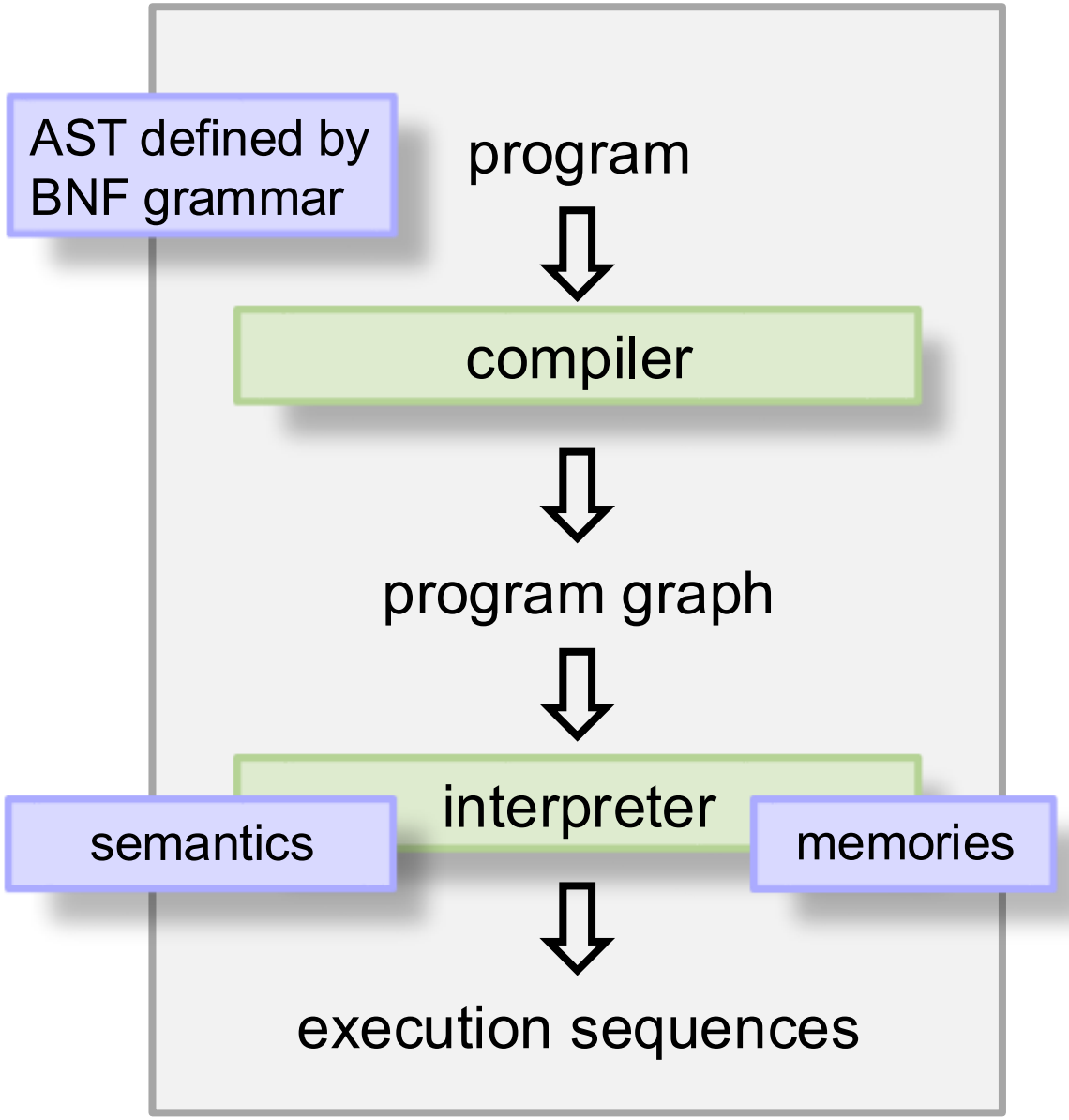


Wrap-Up: Program Semantics

- **Goal:** formalize the behavior of programs mathematically
- **Why?** Foundation for transforming and reasoning about programs
- Compilation and evaluation is formalized via AST traversals



- Recursive definitions ➔ inductive proofs

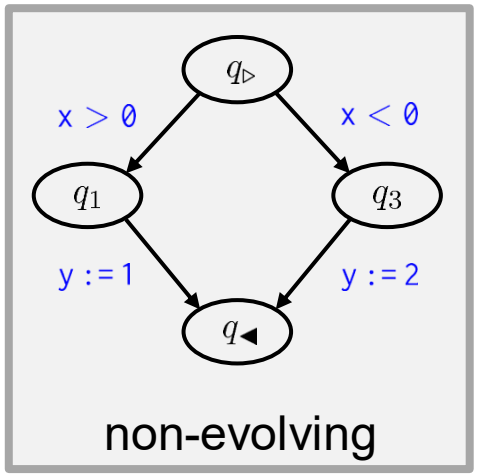


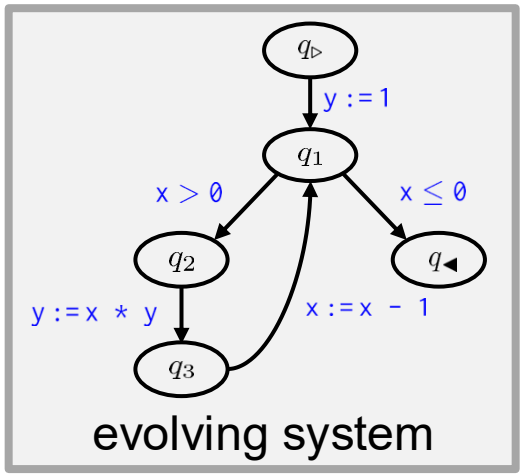
02141 – Computer Science Modelling

EVOLVING-BY-CONSTRUCTION SEMANTICS

Recap: Evolving Systems

An evolving system can always perform execution steps unless it reached a final configuration.





Definition. A program graph and its semantic function constitute an **evolving system** whenever

$$\langle q_\triangleright; \sigma\rangle \overset{\omega}{\Longrightarrow}^* \langle q'; \sigma'\rangle \text{ with } q' \neq q_\blacktriangleleft$$

can always be extended by some $\alpha \in$ **Act** to

$$\langle q_\triangleright; \sigma\rangle \overset{\omega}{\Longrightarrow}^* \langle q'; \sigma'\rangle \overset{\alpha}{\Longrightarrow}^* \langle q''; \sigma''\rangle.$$

Goal: change semantics such that we always produce an evolving system unless some expression is undefined

Exercise

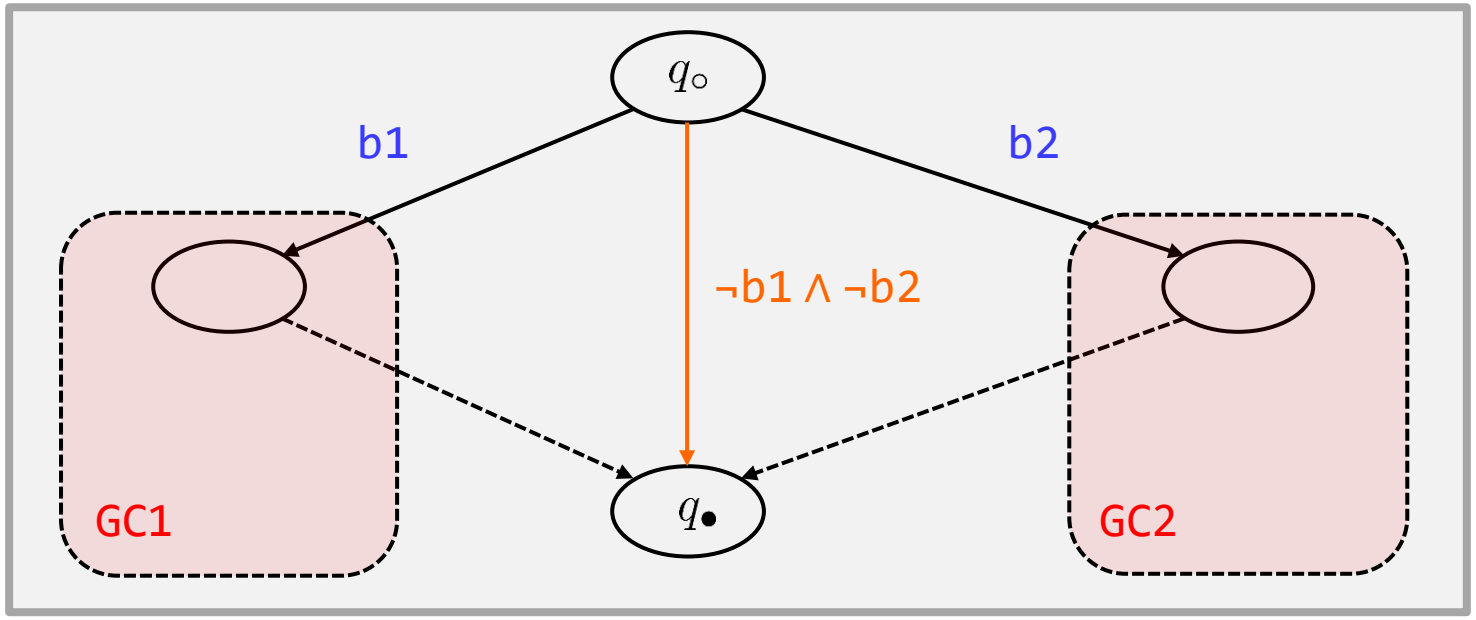
- Extend the following program such that its program graph is an evolving system that admits at least the execution sequences of the original program.

```
if x >= 10 -> y := 10
[] x >= 5 -> y := 5
[] x >= 0 -> y := 0
fi
```

- Explain why the program below describes an evolving system.

```
do x >= 10 -> y := 10
[] x >= 5 -> y := 5
[] x >= 0 -> y := 0
od
```

An Evolving Semantics



$$\textbf{edges}_{\mathsf{e}}(q_\circ \rightsquigarrow q_\bullet)[\![\texttt{if GC fi}]\!] \;=\; \textbf{edges}_{\mathsf{e}}(q_\circ \rightsquigarrow q_\bullet)[\![\texttt{GC}]\!] \cup \{\,(q_\circ, \mathsf{done}[\![\texttt{GC}]\!], q_\bullet)\,\}$$

$$\begin{aligned}
\mathsf{done}[\![\texttt{b -> C}]\!] &= \neg\texttt{b} \\
\mathsf{done}[\![\{\,\texttt{GC}_1\,\}\ [\,]\ \{\,\texttt{GC}_2\,\}]\!] &= \mathsf{done}[\![\texttt{GC}_1]\!] \;\wedge\; \mathsf{done}[\![\texttt{GC}_2]\!]
\end{aligned}$$